# CIVICGRID AI

Smart Public Infrastructure & Complaint Resolution Platform

**Gamified Civic Intelligence Model**

# MASTER AI CONTEXT + BUILD BRIEF

A single handoff document for the team's AI coding agents, design agents, documentation agents and future project chats.

---

**Purpose:** give an AI agent a stable, comprehensive project context so it does not repeatedly ask basic questions, drift from agreed decisions, or expand scope during the hackathon.

**Event context:** LT HackFest 2026 - International Level Online Hackathon - Open Innovation. This document incorporates the official uploaded guidelines plus the team's current project decisions.

**Important memory rule:** an AI cannot literally guarantee permanent, pin-to-pin memory of every future chat unless the product provides persistent memory/context. This PDF is therefore the project's authoritative portable context. The agent should treat it as the working source of truth, preserve it, and update its project-memory files rather than pretending it remembers unavailable messages.

Version 1.0 | 12 August 2026

# 1. HOW TO USE THIS FILE WITH AN AI CODING AGENT

Upload this PDF at the beginning of a fresh agent session and give the agent the instruction below. Put the same rules into the repository as **PROJECT-CONTEXT.md**, **AI-DEVELOPMENT-RULES.md**, and the relevant Kiro/agent steering file so the context survives across sessions.

## Recommended first message:

```
READ THIS DOCUMENT COMPLETELY BEFORE MAKING CHANGES.

Treat this file as the authoritative project context for CivicGrid AI.
Do not invent requirements that are not in this document.
Do not repeatedly ask me for information already specified here.
Do not silently change the product strategy, tech stack, architecture, gamification rules, or demo flow.

You are an AI-assisted member of a 4-person vibe-coding team.
We have a hard hackathon submission deadline and limited build time.
Optimize for a working, polished vertical slice and a 2-3 minute live demo.

When something is ambiguous:
1. Prefer the smallest implementation that satisfies the documented goal.
2. Keep the implementation reversible and modular.
3. State the assumption briefly in your response.
4. Never expand scope unless explicitly instructed.

Before coding:
- inspect the repository;
- inspect existing docs and agent rules;
- identify which teammate/module owns the relevant files;
- reuse existing components and types.

After coding:
- run the safest available typecheck/build/lint/test command;
- summarize changed files, tests, risks and any unresolved issues;
- never claim a feature works without evidence.

Keep all AI-assisted code understandable enough for the team to explain in a judge Q&A.
```

**Pin-to-pin context handling rule:** the agent should not claim that it remembers chats that are not present in its context. Instead it must rely on this document, repository files, commit history and explicitly supplied new instructions. When a new decision supersedes an old one, record the decision in the project context and note what changed.

# 2. LT HACKFEST 2026 - HARD EVENT CONSTRAINTS

These constraints are derived from the official 4-page event document uploaded by the team. Treat them as non-negotiable unless the organizers publish an update. The official document states that Round 1 is Project Submission & Live Demo, the submission deadline is 12 August 2026 at 11:59 PM IST, and the live demos are scheduled for 13-15 August 2026 with a 2-3 minute duration per team. ■filecite■turn0file0■L5-L10■

| Constraint | Official requirement / project implication |
|---|---|
| Round 1 | Project Submission & Live Demo |
| Submission deadline | 12 August 2026 - 11:59 PM IST |
| Round 1 live demo | 13, 14 & 15 August 2026 |
| Demo duration | 2-3 minutes per team |
| Problem format | Open Innovation - team chooses its own problem statement |
| Allowed problem bases | Real-world problem, industry challenge, social issue, business problem, technology challenge or innovative opportunity |
| Expected explanation | Clearly explain the problem, practical relevance, solution, innovation, originality and technical implementation |
| Round 2 | Technical Presentation on 22 & 23 August 2026 if shortlisted |
| Round 2 evaluation | Overview, own problem statement, solution, architecture, stack, innovation, business impact/scalability, future scope, presentation and Q&A; |
| Grand Finale | 30 August 2026 if finalist |
| Final evaluation | Complete demonstration, technical excellence, innovation/creativity, real-world impact, scalability/business potential, UX, functionality, presentation and Q&A; |

The official document explicitly permits AI tools, APIs, frameworks, libraries and open-source technologies, but requires participants to understand and demonstrate the resulting project; judges may ask about AI/external tools. ■filecite■turn0file0■L92-L109■

Intellectual-property responsibility also remains with the team for third-party software, datasets, APIs, images and other resources. ■filecite■turn0file0■L118-L135■

# 3. EVENT LOGISTICS THAT AFFECT THE DEMO

LT HackFest 2026 is fully online through Google Meet. The team must join at least 10 minutes early, use registered team details, have stable internet, working microphone/camera, and be ready to share the screen. Technical issues should be communicated immediately. ■filecite■turn0file0■L78-L91■

| Risk | Team action |
| --- | --- |
| Internet loss | Keep a local copy of the project, local demo data and screenshots/video backup where permitted. |
| Deployment failure | Have a locally runnable fallback and keep the final demo build on the primary laptop. |
| Screen sharing issue | Test browser/app sharing before the assigned slot; know which window to share. |
| AI service outage | Keep AI classification behind a provider interface with deterministic demo fallback. |
| Time pressure | Use a 2-3 minute demo script; do not narrate implementation details that are not visible. |
| Judge AI questions | Prepare a one-page AI usage explanation: where AI is used, where deterministic logic is used, and why. |

# 4. PROJECT IDENTITY - THE SINGLE SOURCE OF TRUTH

**Project name:** CivicGrid AI - Smart Public Infrastructure & Complaint Resolution Platform

**Model:** Gamified Civic Intelligence Platform

**Core tagline:** Report. Earn. Compete. Improve your city.

**One-sentence pitch:** CivicGrid AI lets citizens report public infrastructure issues through a fast, gamified workflow while AI classifies and routes issues to the right department, tracks resolution, and rewards verified civic contribution.

**Strategic differentiation:** CivicGrid is not just a complaint portal. Its central innovation is the combination of AI-assisted civic triage, transparent workflow management and a reputation-based gamification loop that rewards useful participation rather than raw complaint volume.

**Primary outcome:** improve the feedback loop between citizens and organizations responsible for public infrastructure.

# 5. PROBLEM STATEMENT

**Problem:** Citizens encounter potholes, garbage accumulation, water leaks, blocked drains, flooding, damaged roads, broken streetlights and similar infrastructure issues, but reporting and follow-up can be fragmented. On the authority side, unstructured reports, duplicates, manual routing and limited analytics can make prioritization and accountability harder.

**Open Innovation framing:** The project addresses a real-world/social/business/technology problem using AI, web application development, automation, mapping and gamification - all domains explicitly compatible with the event's Open Innovation format. ■filecite■turn0file0■L9-L37■

## Problem statement for submission:

```
Design and develop an AI-assisted, gamified civic issue management platform that enables citizens to
report public infrastructure problems using photos, text and optional voice input; automatically
classifies and routes those issues to the appropriate department; provides transparent status tracking and
workforce workflows; and increases sustained civic participation through reputation, XP, missions, badges
and community leaderboards.
```

**Success criteria for the hackathon:** a judge can create or simulate one issue, see an AI classification, observe routing and resolution, and see the gamification state change in the same demonstration.

# 6. USERS AND END-TO-END FLOW

| Role | Primary goals | MVP permissions |
|---|---|---|
| Citizen | Report, track, verify, earn, compete | Create/view own and nearby issues; earn XP; view leaderboard |
| Municipal/Admin | Triage, assign, monitor | View all issues; assign department/worker; update statuses |
| Field Worker | Execute and prove resolution | View assigned tasks; change status; add resolution evidence |
| Supervisor | Monitor team performance | Review assignments, SLA state and resolution evidence |
| System Admin | Configure platform | Seed data, missions, badges, categories and access control |

## Canonical demo flow:

```
Citizen
-> Report issue
-> AI classification
-> Duplicate check
-> Priority + department routing
-> Admin dashboard
-> Worker assignment
-> Worker resolution
-> Citizen confirmation / system verification
-> XP + reputation + badge
-> Ward leaderboard update
```

# 7. GAMIFICATION MODEL - CORE PRODUCT DIFFERENTIATOR

The system must reward verified civic value, not complaint quantity. This is both a product principle and an anti-abuse control.

| Mechanic | MVP rule |
|---|---|
| XP | Verified issue +20; quality evidence +10; community verification +10; resolved issue +30; mission completion +50 |
| Levels | Citizen -> Reporter -> Guardian -> Community Captain -> City Ranger -> Civic Ambassador -> Urban Champion -> Civic Legend |
| Reputation | Trust score based on useful/accurate reports, verification quality, resolution confirmations and spam/rejection history |
| CivicCoins | Demo virtual currency for future rewards; no real-money economy in MVP |
| Badges | First Report, Road Guardian, Community Hero |
| Mission | One active mission: complete 3 verified road-related contributions |
| Leaderboard | Citizen top-5 and ward top-3 views |
| Anti-spam | Duplicate reports and rejected/spam activity must not receive full rewards |

**Product philosophy:** The gamification layer should make civic contribution feel measurable, social and rewarding without turning the app into a childish game.

# 8. 4-HOUR MVP SCOPE - WHAT WE ARE ACTUALLY BUILDING

**Critical reality:** the team has approximately four hours before submission. Therefore the target is a polished vertical slice, not the full production platform.

## Must work:

• Citizen landing/dashboard

• Report issue form

• Photo/seeded evidence

• AI classification result: category, severity, confidence, department

• Duplicate/nearby issue warning

• Map or map-like issue visualization

• Admin issue queue

• Assignment to worker/department

• Worker status update and resolution

• XP/reputation/badge update

• Citizen and ward leaderboard

• Responsive polished UI

**Do not build today:** native mobile apps, real government integrations, payment gateway, real-money rewards, custom ML training, full predictive-maintenance engine, complex social networking, large-scale moderation, or a production multi-city control plane.

# 9. AI-FIRST TOOLCHAIN

The team is made up of four vibe coders and intends to rely heavily on free AI-assisted tools. Use each tool for a specific role.

| Tool | Role in this project | Current verified note |
|---|---|---|
| VS Code | Primary shared editor and Git client | Use as the main coding environment for all four members. |
| Kiro | Primary AI coding/specification/agent tool when the team uses Kiro | Kiro Free currently lists $0/month, 50 credits, access to open-weight models and Claude Sonnet 4.5. Credits are consumed by prompts/spec tasks, so conserve them. ■cite■turn790701search0■turn790701search3■ |
| Google Stitch | High-fidelity UI generation, design iteration and handoff direction | Google describes Stitch as an AI-native design canvas for high-fidelity UI and collaborative iteration; current updates include export paths toward developer tooling/Antigravity. ■cite■turn790701search5■turn790701search8■ |
| GitHub | Single source of truth, repo, branches, pull requests | GitHub Free allows unlimited collaborators on personal-account public/private repositories. ■cite■turn790701search7■ |
| Supabase | Auth, Postgres, Storage, Realtime | Free plan is currently $0 with stated quotas; free projects pause after 1 week of inactivity. ■cite■turn790701search4■ |
| Vercel | Fast web deployment if suitable | Hobby is currently $0/month, but it is intended for personal/non-commercial use; obey its current terms. ■cite■turn790701search2■ |
| Leaflet + OpenStreetMap | Map visualization without a paid map SDK | Use conservative demo traffic and respect the relevant tile/data policies. |

**Free-cost policy:** The MVP target is ■0 mandatory spend for software, API calls and hosting. Free-tier quotas and provider terms are not guarantees of unlimited production use. If a service starts demanding payment or becomes unavailable, switch to a local/mock adapter instead of changing the project objective.

# 10. VS CODE + GIT COLLABORATION STANDARD

**Primary environment:** VS Code. All four developers work from one GitHub repository.

**Repository:** civicgrid-ai (private during development unless the event requires public visibility). GitHub Free supports adding collaborators to personal repositories. ■cite■turn790701search7■

**Suggested VS Code extensions:** GitHub Pull Requests and Issues, ESLint, Prettier, Tailwind CSS IntelliSense, Error Lens, optional Live Share, optional GitLens. Do not overload VS Code with many competing AI extensions.

## Branching:

```
main -> stable/demo branch
feature/citizen -> citizen reporting UI
feature/admin -> government dashboard/workflow
feature/ai -> AI/duplicate/provider layer
feature/gamification -> XP/reputation/badges/leaderboards
```

**Collaboration rule:** one person owns each feature area. Do not let multiple agents rewrite the same shared files concurrently. No force-push. No committing secrets. Pull before starting work. Push small coherent commits.

# 11. MASTER PROMPT FOR ANY AI CODING AGENT

Paste this after uploading the PDF. It is intentionally written as a persistent project contract.

```
You are now an engineering agent on the CivicGrid AI project.

FIRST: read and internalize the uploaded "CivicGrid AI - Master AI Context and LT HackFest Build Brief"
completely.

SOURCE-OF-TRUTH ORDER:
1. Official LT HackFest guidelines included/referenced in the PDF
2. This project context PDF
3. Repository files, current database schema and current working code
4. New explicit user/team instructions
5. Your own implementation choices

Never invent requirements that conflict with the above.

MEMORY / CONTEXT RULE:
You must not claim to remember unavailable conversations. Preserve continuity by reading this PDF plus
repository context files, Git history, and current code. When a new decision is made, propose a one-line
update for PROJECT-CONTEXT.md and respect that decision in later tasks.

TEAM:
4 vibe coders. AI-assisted development is expected and permitted by the event.
Editor: VS Code.
Repository: one shared GitHub repo.
Target cost: ■0 mandatory spend for MVP.

PROJECT:
CivicGrid AI - Smart Public Infrastructure & Complaint Resolution Platform
Gamified Civic Intelligence Model.

CORE VERTICAL SLICE:
Citizen report -> AI classify -> duplicate check -> route -> admin assign -> worker resolve ->
XP/reputation/badge -> leaderboard.

TIME:
The immediate submission has a hard deadline. Optimize aggressively for demo reliability and visual
quality. Do not build nice-to-have features unless the core path already works.

ENGINEERING:
- TypeScript
- React/Next.js
- Tailwind
- Supabase/Postgres where available
- Leaflet/OpenStreetMap for map visualization
- Provider-agnostic AI interface
- Local/mock fallback for AI
- No secrets in Git
- Small, reversible changes
- Reuse existing components
- Run build/typecheck/tests after meaningful changes

BEFORE CHANGING CODE:
1. inspect repository;
2. identify module ownership;
3. locate existing types/components/services;
4. state the files you will modify if the change is cross-cutting.

DO NOT:
- rewrite the whole application;
- introduce unnecessary dependencies;
- change unrelated modules;
- break working routes;
- hardcode API keys;
- add a paid API as a requirement;
- silently alter the gamification rules;
- claim untested functionality works.

AFTER EACH TASK:
Report:
- what changed;
- files changed;
- tests/build run;
- known limitations;
```

```
- next smallest safe step.

JUDGE-READINESS:
The team must be able to explain every major AI-assisted feature in a short Q&A.
```

# 12. ARCHITECTURE AND MODULE CONTRACTS

```
Frontend
Next.js / React / Tailwind
|
+---- Citizen UI
+---- Admin UI
+---- Worker UI
+---- Gamification UI
|
Service layer
|
+---- issueService
+---- workflowService
+---- aiClassifier
+---- duplicateDetector
+---- gamificationEngine
+---- notificationAdapter
|
Data / Infrastructure
|
+---- Supabase Auth
+---- PostgreSQL
+---- Storage
+---- Realtime (optional)
+---- Leaflet / OpenStreetMap
```

**Key architectural rule:** the AI classifier is an interface. The MVP can use a deterministic/mock provider; a real local/open model or external API can be plugged in later without rewriting the UI or workflow.

## Example AI result:

```
{
"category": "Pothole",
"severity": "High",
"confidence": 0.94,
"department": "Roads",
"reason": "Road-surface damage detected in the supplied evidence"
}
```

# 13. MVP DATA MODEL

| Table | Minimum fields | Notes |
|---|---|---|
| profiles | id, name, ward, role, xp, level, reputation, civic_coins | Auth identity + gamification state |
| issues | id, reporter_id, category, description, image_url, lat, lng, severity, status, department, ai_confidence, duplicate_of | Core civic ticket |
| issue_events | id, issue_id, actor_id, event_type, note, created_at | Immutable-ish timeline/audit trail |
| assignments | id, issue_id, worker_id, assigned_at, accepted_at, completed_at | Work execution |
| missions | id, title, rule_type, target, xp_reward, coin_reward, active | Mission definitions |
| mission_progress | id, user_id, mission_id, progress, completed_at | Progress state |
| badges | id, name, description, icon | Badge catalog |
| user_badges | user_id, badge_id, earned_at | Achievements |
| rewards | id, title, description, coin_cost, active | Future/demo reward catalog |

**Seed data:** pre-populate 8-12 issues across 3-4 wards so the admin dashboard and map look alive before the live demo.

# 14. FOUR-PERSON TEAM EXECUTION MODEL

| Member | Owns | Do not touch without coordination |
|---|---|---|
| A - Integration Lead | Repo, app shell, shared types, database, merge stability | Other members' feature-specific UI unless necessary |
| B - Citizen/UX | Stitch output, citizen dashboard, report flow, issue details | Admin/worker feature internals |
| C - Admin/Workflow | Admin queue, department routing, assignments, worker resolution | Gamification engine internals |
| D - AI/Gamification/QA | AI adapter, duplicate logic, XP/reputation/badges/leaderboards, seeded data, test checklist | Shared layout unless coordinated |

## Four-hour schedule:

| Time | Goal |
|---|---|
| 00:00-00:15 | Freeze scope, confirm branch ownership, read this brief, confirm demo story |
| 00:15-00:45 | Bootstrap app/repo/database; generate Stitch visual direction; create seed data |
| 00:45-01:30 | Build citizen report + AI result + database write |
| 01:30-02:15 | Build admin queue + routing + worker assignment |
| 02:15-02:45 | Build worker resolution + status timeline |
| 02:45-03:10 | Build XP/reputation/badge/leaderboards; duplicate warning |
| 03:10-03:30 | Connect vertical slice end-to-end; fix core bugs |
| 03:30-03:45 | UI polish, loading/error states, responsive checks |
| 03:45-04:00 | Build/deploy/local backup, rehearse 2-3 minute demo, stop adding features |

# 15. TESTING REQUIREMENTS MAPPED TO THE PROJECT

The project also has a software-engineering/documentation requirement: SRS, design, SCM, risk, CASE-tool design, unit/integration testing, white-box testing and black-box testing. These artifacts are important for documentation even though they are not all MVP features.

| Testing area | CivicGrid example | Evidence |
|---|---|---|
| Unit | XP calculation, reputation calculation, category validation, status transition validation | Test table + implementation test output |
| Integration | Report form -> issue DB -> admin queue; assignment -> worker; resolution -> reward | End-to-end test case |
| White-box | Reward branches, duplicate decision branches, workflow transitions | Statement/branch/path analysis + cyclomatic complexity for key functions |
| Black-box | Valid/invalid category, severity boundaries, empty description, duplicate/non-duplicate, state transitions | Equivalence partitions, boundary values, decision tables, state-transition tests |
| Regression | Core report flow after UI/AI changes | Retest checklist |

# 16. CASE TOOL AND SOFTWARE DOCUMENTATION PLAN

**Recommended CASE/design tool:** diagrams.net (Draw.io). Use it for the minimum set of design-phase artifacts because the team has a four-hour build constraint and the diagrams need to match the code.

| Required document | Minimum content |
|---|---|
| Problem statement | Background, pain, gap, stakeholders, solution statement, measurable objectives |
| SRS | Functional requirements, non-functional requirements, actors, use cases, constraints, acceptance criteria |
| HLD | Architecture, components, integrations, deployment view |
| LLD | Module contracts, key services, data model, workflow/state model |
| SCM | Repo structure, branching, change control, versioning, release process |
| Risk management | Risk register, probability/impact, mitigation, contingency |
| CASE-tool record | Tool selection, why selected, diagrams created, screenshots |
| Testing package | Unit, integration, white-box, black-box test cases and results |

# 17. RISK REGISTER FOR THIS VIBE-CODING HACKATHON

| Risk | Impact | Mitigation |
|---|---|---|
| AI agent changes unrelated code | High | Module ownership, small branches, review diffs before merge |
| Kiro credits exhausted | High | Use concise prompts; use other available tools or local/manual implementation; never make project dependent on one agent |
| Free cloud quota/availability | High | Seed data + local/mock adapters + local fallback |
| Model output inconsistent | High | Keep a deterministic provider fallback and a fixed response schema |
| Scope creep | Very high | Core vertical slice only; freeze features at ~3 hours |
| Broken merge | High | Pull before work, small commits, no force push, integration lead controls main |
| Demo internet failure | High | Local demo + preloaded data + screen recording backup where allowed |
| Judges question AI usage | Medium | Document exact AI use and deterministic logic; team must understand code |
| Third-party license issue | Medium | Keep a dependency/source register and confirm rights before submission |

# 18. READY-TO-PASTE AGENT PROMPTS

## Repo bootstrap prompt:

```
Read the CivicGrid AI project context first. Inspect this repository. If it is empty, initialize a Next.js + TypeScript + Tailwind project using the simplest reliable setup. Create the required folder structure, .env.example, README, shared types, and the minimal Supabase-compatible service layer. Do not implement admin, worker, AI or gamification features yet. Stop after verifying the app builds.
```

## Citizen feature prompt:

```
Implement only the citizen reporting vertical slice described in the project context. Reuse existing UI and types. Build the report form, evidence upload/preview, location, AI result card, duplicate warning and issue submission. Do not modify admin or worker feature logic. Run typecheck/build after the change.
```

## Admin/workflow prompt:

```
Implement only the admin queue, department routing and worker assignment flow. Use the existing issue types and service interfaces. Build a visually strong dashboard with priority, severity, SLA indicator, department and assignment actions. Do not rewrite citizen UI. Test the entire status transition path.
```

## Gamification prompt:

```
Implement only XP, reputation, badges, mission progress and leaderboards. Use deterministic rules from the project context. Never award full XP for duplicate/spam reports. Add unit tests for the reward calculation branches. Keep the UI youthful but professional.
```

## Bug-fix prompt:

```
Do not rewrite the feature. Reproduce the bug, identify the smallest root cause, make the smallest safe fix, run the relevant build/typecheck/test, and report the exact files changed. Preserve all existing functionality.
```

# 19. 2-3 MINUTE LIVE DEMO SCRIPT

The official Round 1 live demo is only 2-3 minutes, so the team must not attempt a long feature tour.
■filecite■turn0file0■L5-L8■

| Time | Narration + action |
|---|---|
| 0:00-0:20 | Problem: 'Citizens see civic issues but the feedback loop is fragmented, and authorities receive unstructured/duplicate reports.' |
| 0:20-0:45 | Open CivicGrid citizen view. Report a pothole with evidence. Show the AI result: Pothole / High / Roads / confidence. |
| 0:45-1:10 | Show duplicate check and issue appearing in the dashboard/map. Explain automatic routing. |
| 1:10-1:35 | Switch to admin/worker. Assign the issue, mark In Progress, then Resolved with evidence. |
| 1:35-1:55 | Return to citizen. Show XP, reputation, badge and ward leaderboard update. |
| 1:55-2:20 | State innovation: AI triage + transparent workflow + verified gamification. State scale: municipalities, campuses, industrial communities. |
| 2:20-2:50 | Close with business/scalability and invite Q&A.; |

**Demo rule:** never wait for AI generation live if it is risky. Preload a deterministic or cached AI result and explain the architecture. Reliability is more important than theatrical randomness.

# 20. JURY Q&A; - ANSWERS THE TEAM MUST UNDERSTAND

| Question | Core answer |
|---|---|
| What is innovative? | The system combines AI-assisted triage and routing with a trust-aware gamification loop that rewards verified civic contribution, not complaint volume. |
| Why gamification? | Traditional reporting can have weak retention. Missions, reputation, badges and community competition create recurring participation while reputation/verification reduce spam. |
| What prevents fake complaints? | Duplicate detection, evidence quality, reputation, community verification and moderation rules. |
| Is all of this AI? | No. AI is used where it adds value - classification/analysis. Workflow, rewards, state transitions and core business rules remain deterministic for reliability. |
| Why free tools? | The MVP is designed around free/open-source/free-tier components. Paid services are not mandatory for the demo. |
| Can this scale? | The modules are separated into UI, services, AI adapters, gamification and data. The same model can be configured for municipalities, campuses and private communities. |
| How will you make money? | B2G SaaS, enterprise civic-operations deployments, analytics and sponsored community reward programs are future options. |
| What would you build next? | Multilingual voice, stronger computer vision, predictive maintenance, official government integrations, richer rewards and a multi-tenant control plane. |

# 21. PROJECT MEMORY FILES TO KEEP IN THE REPOSITORY

To keep AI agents consistent across chats/tools, commit these files to GitHub:

| File | Purpose |
|---|---|
| PROJECT-CONTEXT.md | Portable master summary of product, constraints and current decisions |
| AI-DEVELOPMENT-RULES.md | Rules every AI agent must follow |
| ARCHITECTURE.md | Current architecture and module contracts |
| DECISIONS.md | Short dated record of decisions that supersede earlier assumptions |
| DEMO-SCRIPT.md | Current 2-3 minute demo flow |
| TEST-RESULTS.md | Current test status and known issues |
| THIRD-PARTY.md | Dependencies, licenses, datasets and assets used |
| docs/ | SRS, HLD, LLD, SCM, Risk and test documents |

When a project decision changes, update **DECISIONS.md** and **PROJECT-CONTEXT.md**. This is more reliable than asking an AI to 'remember' a chat that may no longer be available.

# 22. CURRENT DECISION LOG - DO NOT LOSE THESE

| Decision | Current state |
|---|---|
| Project name | CivicGrid AI - Smart Public Infrastructure & Complaint Resolution Platform |
| Product model | Gamified civic intelligence / citizen-to-authority workflow |
| Team | 4 members; all are vibe coders / AI-assisted developers |
| Editor | VS Code |
| AI development | Heavy use of Kiro/AI agents; Google Stitch for design; other free AI tools may be used as backup |
| Repository | One shared GitHub repository for all four members |
| Development cost | ■0 mandatory spend for MVP |
| Primary stack | Next.js/React + TypeScript + Tailwind + Supabase-compatible backend |
| Map | Leaflet/OpenStreetMap |
| AI architecture | Provider-agnostic classifier with deterministic/local/mock fallback |
| Gamification | XP + reputation + CivicCoins + badges + missions + citizen/ward leaderboards |
| Demo | 2-3 minute end-to-end vertical slice |
| Top priority | Stable, visually polished, explainable demo over feature quantity |
| Hard rule | At ~3 hours, stop adding features; use remaining time for integration, QA, deployment/backup and rehearsal |

# 23. ANTI-DRIFT RULES FOR FUTURE AI CHATS

• Do not turn CivicGrid into a generic grievance portal. The gamification/reputation loop is central.

• Do not replace the core civic workflow with a chatbot-only concept.

• Do not make paid AI calls a hard dependency for the MVP.

• Do not build a native mobile app during the four-hour submission window.

• Do not add unrequested technologies just because an AI recommends them.

• Do not claim real municipal integration unless it actually exists.

• Do not fabricate accuracy metrics, government partnerships, user counts, cost savings or deployment claims.

• Do not reward raw complaint spam.

• Do not conceal AI assistance from judges; the official rules allow AI tools, but the team must understand and demonstrate the project. ■filecite■turn0file0■L97-L105■

• Do not use third-party images/datasets/code without tracking the rights/permissions responsibility required by the event. ■filecite■turn0file0■L130-L135■

**When in doubt:** choose the simplest implementation that makes the core demo stronger and can be explained in one minute.

# 24. CURRENT TOOL VERIFICATION NOTES

These notes were checked against current web sources on 12 August 2026 and should be rechecked if the project continues beyond the hackathon because pricing/free tiers can change.

Kiro Free is currently listed at $0 with 50 credits; Kiro states credits are consumed by prompts, spec tasks and related agent actions. ■cite■turn790701search0■turn790701search3■

Google Stitch was updated in May 2026 with collaborative design-agent workflows, real-time iteration and export paths toward developer tooling. ■cite■turn790701search5■

Supabase Free is currently listed at $0/month with stated database/storage/user quotas and project pausing after a week of inactivity. ■cite■turn790701search4■

Vercel Hobby is currently listed at $0/month, but Vercel says Hobby is for personal, non-commercial use. Treat it as a hackathon/demo deployment option and check current terms before commercial use. ■cite■turn790701search2■

GitHub Free allows unlimited collaborators on public and private personal repositories, supporting the shared-repo setup described here. ■cite■turn790701search7■

# 25. FINAL PRE-SUBMISSION PREFLIGHT

| Check | Pass condition |
|---|---|
| Repository | All four can clone/pull and the same commit runs successfully |
| Build | No type/build errors on the demo machine |
| Citizen flow | Issue can be created and is visible |
| AI result | Classification card visibly appears |
| Admin flow | Issue is routed and assignable |
| Worker flow | Issue can reach Resolved |
| Gamification | XP/reputation/badge/leaderboard visibly changes |
| Demo data | Enough seeded data for a believable dashboard |
| Fallback | Local/mock AI and local project run are available |
| Presentation | 2-3 minute script rehearsed |
| Rules | Team, originality, AI-use explanation and third-party rights are understood |
| Submission | Files/links accessible before 11:59 PM IST deadline |

**Official submission deadline reminder:** 12 August 2026 - 11:59 PM IST. Only teams submitting on or before that deadline are eligible for Round 1 under the uploaded official guidelines. ■filecite■turn0file0■L46-L47■

## 26. SOURCES AND AUTHORITY

**Primary event source:** LT HackFest 2026 - Official Round-Wise Guidelines, Rules & Regulations (uploaded by the team, 4 pages). Key requirements cited throughout this brief come directly from that document. ■filecite■turn0file0■L1-L8■

**Current tool verification:** Kiro official pricing/billing, Google Labs Stitch posts, GitHub repository collaboration documentation, Supabase pricing, and Vercel pricing were checked on 12 August 2026. These are reference notes, not guarantees of future pricing or availability. ■cite■turn790701search0■turn790701search3■turn790701search5■turn790701search7■turn790701search4■turn790701search2■

**Authority rule for conflicts:** If an AI agent, tutorial, older chat, or this PDF conflicts with a newer official LT HackFest announcement, the official organizer communication wins. Record the change in DECISIONS.md and update PROJECT-CONTEXT.md.

# END OF MASTER CONTEXT

# APPENDIX A - AUTOMATED GITHUB + PERMISSION-CONTROLLED AI WORKFLOW

This appendix is added to the CivicGrid AI master context so the same AI agent that builds the product can also initialize, commit, push, branch and open pull requests - while keeping destructive or production-impacting actions behind human approval.

**Desired operating model:** automate the boring Git work, but keep a human gate on main-branch changes, destructive repository operations, security changes and anything that could erase another teammate's work.

The official LT HackFest rules allow AI tools, APIs, frameworks, libraries and open-source technologies, but require the participating team to understand and demonstrate the project; judges may ask about AI/external tools.
■filecite■turn0file0■L97-L105■

# A1. FIRST-TIME REPOSITORY BOOTSTRAP

The first AI agent run should be allowed to prepare the project for GitHub, but it must stop before creating/publishing the remote unless GitHub authentication is already available and the human has explicitly authorized the repository creation step.

```
FIRST-RUN GIT/GITHUB SETUP INSTRUCTION

You are the Git and project-bootstrap agent for CivicGrid AI.

1. Inspect the current folder.
2. Detect whether Git is initialized.
3. If not, run git init.
4. Inspect the repository before adding files.
5. Create a safe .gitignore.
6. Never commit .env, credentials, private keys, tokens or API secrets.
7. Ensure these files exist:
- README.md
- PROJECT-CONTEXT.md
- AI-DEVELOPMENT-RULES.md
- ARCHITECTURE.md
- DECISIONS.md
- AI-GIT-RULES.md
8. Create the initial commit.
9. Check whether origin already exists.
10. If origin does not exist, prepare for GitHub publishing.

GITHUB PUBLISHING:
- Use the already authenticated VS Code/GitHub environment.
- Do not ask the user to paste a password or token into chat.
- If GitHub authentication is unavailable, stop and report exactly what must be authenticated.
- When explicitly authorized, create a repository named civicgrid-ai.
- Prefer PRIVATE visibility during development.
- Add origin.
- Push initial main.
- Verify the remote and pushed commit.
- Report the repository URL.

Never:
- force push;
- delete the repository;
- delete main;
- rewrite published history;
- change visibility unexpectedly;
- change collaborators/security settings without explicit permission.
```

**Why this matters:** the first run becomes a repeatable bootstrap procedure instead of a manual Git tutorial for every team member.

# A2. AUTOMATION PERMISSION MATRIX

| Operation | AI may do automatically? | Rule |
|---|---|---|
| git status / diff / log | Yes | Safe read-only operations |
| git fetch / pull | Yes | Allowed on the current working branch |
| create feature branch | Yes | Use module-specific branch names |
| git add / commit | Yes | Review diff first; never stage secrets |
| push feature branch | Yes | Allowed after build/typecheck/test/secret check |
| create/update PR | Yes | PR targets main; no auto-merge |
| push main | No | Human approval required |
| merge PR | No | Human approval required |
| force push | Never | Not permitted |
| hard reset / destructive clean | No | Human approval required |
| delete branch | No | Human approval required |
| delete repository | Never | Not permitted to agent |
| change repo visibility | No | Human approval required |
| change collaborators | No | Human approval required |
| change security/settings | No | Human approval required |
| create/expose access tokens | Never | Not permitted |

**Principle:** the agent receives enough access to build and collaborate, but not enough access to erase the team's source of truth.

# A3. FOUR-DEVELOPER BRANCH MODEL

```
main
|
+-- feature/citizen
+-- feature/admin
+-- feature/ai
+-- feature/gamification
|
+--> Pull Request --> main
|
+--> Human approval required
```

| Team member | Branch | Primary ownership |
|---|---|---|
| Member 1 - Integration Lead | feature/integration | App shell, shared types, DB, merge stability |
| Member 2 - Citizen/UX | feature/citizen | Citizen dashboard, report flow, issue details |
| Member 3 - Admin/Workflow | feature/admin | Admin queue, routing, worker assignment/resolution |
| Member 4 - AI/Gamification/QA | feature/ai-gamification | AI adapter, duplicate logic, XP/reputation/badges/leaderboards |

**Shared-file rule:** package.json, database migrations, shared types and design tokens should have one coordination owner. AI agents must inspect the current diff before modifying a shared file.

# A4. AUTOMATED FEATURE WORKFLOW

After the repository exists, the desired AI behavior is:

```
User request
|
AI reads project context
|
Inspect repository + current branch
|
Create feature branch
|
Implement only requested module
|
Run typecheck / build / tests
|
Inspect git diff + secret check
|
Commit
|
Push feature branch
|
Create/update Pull Request
|
STOP - human reviews/merges main
```

This allows the team to tell an agent things such as **"Implement the citizen reporting flow according to PROJECT-CONTEXT.md"** and have Git automation happen as part of the normal workflow without giving the agent unrestricted control over main.

# A5. MASTER GIT AUTOMATION PROMPT FOR THE AI AGENT

```
You are the Git-aware engineering agent for CivicGrid AI.

SOURCE OF TRUTH:
1. Official LT HackFest rules
2. CivicGrid AI Master AI Context
3. PROJECT-CONTEXT.md
4. AI-DEVELOPMENT-RULES.md
5. AI-GIT-RULES.md
6. Current repository code and Git history
7. Explicit new user instructions

GIT BEHAVIOR:
- Always inspect git status before editing.
- Pull latest main before starting a new feature.
- Create a feature branch for feature work.
- Never work directly on main unless explicitly instructed for a non-code maintenance action.
- Review git diff before staging.
- Run typecheck/build/tests when available.
- Check for secrets before commit/push.
- Make small meaningful commits.
- Push only the feature branch automatically.
- Create/update a Pull Request against main automatically.
- Do not merge the PR.
- Do not push directly to main automatically.

REQUIRE HUMAN APPROVAL FOR:
- push to main
- merge
- force push
- reset --hard
- git clean -fd
- branch deletion
- repository deletion
- visibility changes
- collaborator changes
- GitHub security/settings changes

NEVER:
- expose tokens;
- ask the user to paste secrets into chat;
- commit .env;
- overwrite another developer's branch;
- rewrite published history;
- claim a push/PR succeeded without verifying it.

BEFORE EVERY AUTOMATED PUSH:
1. git status
2. git diff
3. confirm branch is not main
4. run build/typecheck/test
5. scan staged files for secrets
6. commit
7. push
8. verify remote branch
9. create/update PR
10. report URL and checks

After the PR is created, STOP and wait for human merge approval.
```

# A6. TEAM SETUP INSTRUCTIONS

Each of the four developers should authenticate with their own GitHub account in VS Code. Do not share one person's GitHub credentials among all four members. This preserves commit attribution and makes access removal/recovery easier.

**Repository owner:** create/publish civicgrid-ai once, then add the other three teammates as collaborators.

**Each teammate:** clone the same repository, authenticate to GitHub in VS Code, pull main, create their feature branch, and push only that branch.

```
git clone https://github.com/<OWNER>/civicgrid-ai.git
cd civicgrid-ai
npm install
git switch -c feature/<your-module>
npm run dev
```

**Normal feature completion:** pull main -> implement -> test -> commit -> push feature branch -> open PR -> wait for human merge.

# A7. SECURITY AND CREDENTIAL RULES

| Secret / sensitive item | Rule |
|---|---|
| .env | Never commit. Keep .env.example with variable names only. |
| GitHub PAT / OAuth secret | Never paste into AI chat. Use VS Code/GitHub authentication where possible. |
| Supabase service-role key | Never expose to browser/client code; keep server-side only. |
| Public client key | Only use when the provider explicitly defines it as safe for client use; still document it correctly. |
| API keys | Never commit. Rotate immediately if accidentally exposed. |
| Personal data | Use seeded/demo data for hackathon; do not invent real citizen records. |
| Repository settings | Human-controlled. |

**Agent rule:** if a secret is discovered in a file, do not copy it into output, commit it, or paste it into a chat. Stop the risky operation, tell the human what was found, and suggest safe remediation.

# A8. GITHUB + AGENT OPERATING AGREEMENT

The team agrees that automation should remove repetitive work, not remove accountability.

• The AI agent may prepare and publish feature work when authenticated and authorized.

• The AI agent may create pull requests, but main remains a human-reviewed integration point.

• Every teammate owns a branch/module and is responsible for understanding the AI-generated changes they submit.

• The team will not claim that AI-generated code was manually authored if asked; the event explicitly permits AI-assisted development, but judges may ask how it was used. ■filecite■turn0file0■L97-L105■

• The repository itself becomes part of the project's memory system: code, docs, decisions, test evidence and Git history should tell the story even if an AI chat is lost.

**Final desired behavior:** ask the AI to build; the AI creates a safe branch, implements, tests, commits, pushes, opens a PR, and reports the PR. The human decides whether the change becomes part of main.